Instructions

Close ×

Instructions

In this assignment, you'll implement a tiled version of matrix multiplication to improve cache performance. Tiling breaks large matrices into smaller submatrices (tiles) that fit into the cache, reducing memory access overhead.

How Matrix Multiplication Works

In matrix multiplication, the element at row `i` and column `j` of the result matrix `C` is calculated as the dot product of row `i` of matrix `A` and column `j` of matrix `B`:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} \cdot B_{kj}$$

C, start subscript, i, j, end subscript, equals, sum, start subscript, k, equals, 0, end subscript, start superscript, N, minus, 1, end superscript, A, start subscript, i, k, end subscript, dot, B, start subscript, k, j, end subscript

In C++, since matrices are stored as 1D arrays in row-major order, the element at row `i`, column `j` is accessed with: `C[i * N +j]`

An non-tiled implementation uses three nested loops:

Why Use Tiling?

For large matrices, the non-tiled implementation becomes cache inefficient. As the loops iterate, the CPU keeps fetching data from main memory because the working set doesn't fit in cache.

Tiling (also called blocking) improves cache performance by operating on smaller submatrices (tiles) that are sized to fit into cache. This increases data reuse and reduces cache misses.

Task

Complete the implementation of the following function in `tiled_matrix_mult.cpp`:

```
void tiled_matrix_multiply(double* A, double* B, double* C, int N, int tile_size)
```

Where:

- `A`, `B`, and `C` are pointers to the input and output matrices.
- `N` is the dimensions of the square matrices.
- `tile_size` is the size of the tiles to be used for tiling.

Your function should:

- Divide the matrices into tiles of size `tile_size x tile_size`.
- Multiply the tiles and accumulate results into `C`.
- Handle edge cases where N is not divisible by `tile_size`.

Hints

- Use **three outer loops** to iterate over tiles.
- Use **three inner loops** to multiply elements within each tile.
- Initialize `C` to zeros before accumulating results.
- Use `min(i + tile_size, N)` to avoid out-of-bounds access for edge tiles.
- Start with the outer loops first, then add inner loops for the tile operations.

Compare Performance

The provided code includes a non-tiled implementation. Once you complete your tiled version:

1. Run the program with different `tile_size` values.
2. Observe the execution times for each configuration.
3. Reflect: Do larger tiles result in faster execution times? Why might very small tiles (e.g., `tile_size=1`) perform worse than the non-tiled implementation?

Submission

Submit your updated `tiled_matrix_mult.cpp` with your completed `tiled_matrix_multiply` function.

Mark as completed